TECHNICAL REPORT

Azure-Accelerated Foreground-Masking Optimisation

Linux setup, multiprocessing design, measured performance and operating procedure

SEP and MTObjects | Spike Gate and Toy Objects

Prepared for Gordon Bowser

30 July 2026 | Production run report

Executive summary

The four foreground-masking optimisers were successfully made operational on an Ubuntu Linux virtual machine in Microsoft Azure. Optuna and SEP installed natively from Python packages. MTObjects also worked on Linux after its C sources were compiled into four shared libraries against GNU Scientific Library (GSL). No GPU was required.

Production result: One full run of every optimiser completed on an 8-vCPU Standard_D8ns_v6 VM. All result databases, CSV files, configurations and best-parameter JSON files were downloaded to the desktop PC before the VM was deallocated. The four Azure runs took about 74 minutes of optimiser wall time in total.

- SEP Spike Gate: 80 evaluations; best objective 13.9208; approximately 1 minute 51 seconds.
- SEP Toy Objects: 40 evaluations; best recovery score 0.6513; approximately 1 minute 31 seconds.
- MTObjects Spike Gate: 60 evaluations; best objective 4.9278; approximately 27 minutes 7 seconds.
- MTObjects Toy Objects: 40 evaluations; best recovery score 0.1302; approximately 43 minutes 36 seconds.

The large difference between SEP and MTObjects is genuine: MTObjects performs substantially more work for each image/parameter combination. The toy-object MTObjects run is therefore a reasonable conservative planning case.

1. Scope and system architecture

The project contains four independent Python optimiser programs. They remain separate programs and produce separate Optuna studies and parameter files. Cloud execution changes where and how the image-level scoring is performed; it does not merge the scientific methods.

Technique	Detection method	Optimiser program	Default search
Spike Gate	SEP	optimise_spike_gate_SEP.py	16 initial + 64 TPE
Toy Objects	SEP	optimise_toy_objects_SEP.py	8 initial + 32 TPE
Spike Gate	MTObjects	optimise_spike_gate_MTObjects.py	12 initial + 48 TPE
Toy Objects	MTObjects	optimise_toy_objects_MTObjects.py	8 initial + 32 TPE

A run uses 20 representative galaxy images. Toy-object cases inject six synthetic sources per image (120 objects in total). Spike-gate cases retain only galaxies for which the configured profile analysis finds usable spike samples.

2. Azure account and resource setup

The deployment used the G_Bowser_v1 Azure subscription. The initial free-trial subscription exposed VM sizes but assigned zero usable quota for the required families. Upgrading to Pay-As-You-Go enabled quota management; Microsoft.Compute then had to be registered explicitly before compute quotas appeared.

1. **Create or activate an Azure subscription.** A Microsoft account and an Azure subscription are required. Pay-As-You-Go was necessary here because the free trial was not eligible for the required quota increase.
2. **Set cost controls.** Create a small budget and alerts in Cost Management + Billing. A budget is an alerting control, not a prepaid balance and not a hard spending cap.
3. **Register the compute provider.** Open Subscriptions > G_Bowser_v1 > Settings > Resource providers; select Microsoft.Compute and choose Register.
4. **Confirm regional quota.** Open Usage + quotas, select Provider: Compute and the intended region. Verify that the selected VM family has at least eight vCPUs available.
5. **Create a resource group.** Use a dedicated group (mtobjects-linux_group) so the VM, network interface, public IP, network security group and disks are easy to find and remove together.
6. **Create the VM.** Use Ubuntu Server 24.04 LTS, x64 Gen2, Trusted Launch, an SSH public key and Standard SSD OS storage. The production runs used UK South and an 8-vCPU Standard_D8ns_v6 size.
7. **Secure network access.** Permit inbound TCP 22 only for SSH and preferably restrict the rule to the current public IP. Do not expose application or database ports.
8. **Configure shutdown controls.** Enable an auto-shutdown schedule as a backstop. Always deallocate through Azure after work; merely shutting down inside Linux may leave billable allocation.

3. Verified Linux compatibility

Component	Linux result	Installation or build route	Important note
Optuna	Passed	pip in Python virtual environment	SQLite/RDB study storage worked on Ubuntu.
SEP	Passed	pip package (sep 1.4.x range)	Headless use required no graphical desktop.
MTObjects	Passed	GCC build against libgsl-dev	Four .so libraries compiled and loaded with ctypes.
NumPy/SciPy	Passed	pip wheels	Some nearly identical data produced benign precision-loss warnings.
Astropy	Passed	pip wheel	FITS input read successfully.

Component	Linux result	Installation or build route	Important note
Matplotlib	Passed	pip wheel	Available for report products; cloud optimisation itself was headless.
openpyxl	Passed	pip package	Workbooks were intentionally skipped on non-Windows unless requested.

The MTObjects build is the only non-trivial Linux dependency. The bootstrap installs build-essential, git, libgsl-dev, python3, python3-tk and python3-venv. It then compiles mt_objects.so, maxtree.so, mt_objects_double.so and maxtree_double.so from the upstream sources. A final smoke test imports all Python packages and loads every shared library.

```
sudo apt-get install -y build-essential git libgsl-dev python3 python3-tk python3-venv
python3 -m venv .venv-azure-mtobjects
source .venv-azure-mtobjects/bin/activate
pip install -r "Foreground Masking/Azure/requirements-linux.txt"
bash "Foreground Masking/Azure/bootstrap_mtobjects_ubuntu.sh"
```

4. Data preparation and transfer

Twenty FITS images and their smooth-model inputs were transferred rather than the whole galaxy archive. An Azure-specific CSV manifest preserved the scientific selection while replacing Windows drive-letter paths with absolute Linux paths.

- Linux manifest: /home/azureuser/data/mtobjects-20/manifest/azure_mtobjects_20_manifest.csv
- Repository: /home/azureuser/src/PythonScripts-azure
- MTObjects source/build: /home/azureuser/src/mtobjects
- Virtual environment: /home/azureuser/src/PythonScripts-azure/.venv-azure-mtobjects
- Cloud outputs: /home/azureuser/data/results/full-optimisers/<method>

Path rule: Windows paths such as C:\ or D:\ cannot be used by Linux. Every image and model path in the cloud manifest must resolve on the VM before a long run begins.

5. Parallelisation design

Parallelism is applied within each Optuna trial, across images. Optuna trials themselves remain sequential. This design accelerates the expensive scientific scoring while preserving the deterministic order in which Optuna receives objective values.

- The parent process prepares image cases once and owns the Optuna study and output files.

- A process pool scores independent images concurrently; --workers 8 requests up to eight worker processes.
- On Linux the pool uses fork, so prepared NumPy arrays can be shared copy-on-write instead of being serialised for every trial.
- On Windows the implementation uses spawn for compatibility; worker initialisers receive the required state explicitly.
- pool.map preserves input order. The parent aggregates results in the same galaxy order as serial execution.
- The effective worker count is capped at the number of usable image cases.
- Only the parent writes CSV, JSON and SQLite study outputs, avoiding concurrent file corruption.
- --workers 1 retains the original serial path and is the reference behaviour for reproducibility checks.

Reproducibility evidence: For all three newly parallelised programs, seeded one-trial runs with one and two workers produced exactly matching summary parameters and per-image details after excluding elapsed time. Separate eight-worker smoke tests completed over eight FITS images for all four programs.

6. Production commands

The following pattern was used after activating the environment and setting MTOBJECTS_ROOT:

```
export FOREGROUND_MASKING_PC=Desktop
export MTOBJECTS_ROOT=/home/azureuser/src/mtobjects
MANIFEST=/home/azureuser/data/mtobjects-20/manifest/azure_mtobjects_20_manifest.csv

python "Foreground Masking/optimise_spike_gate_SEP.py" \
  --manifest "$MANIFEST" --max-images 20 --initial-points 16 --max-iter 64 --workers 8

python "Foreground Masking/optimise_toy_objects_SEP.py" \
  --manifest "$MANIFEST" --max-images 20 --initial-points 8 --max-iter 32 --workers 8

python "Foreground Masking/optimise_spike_gate_MTObjects.py" \
  --manifest "$MANIFEST" --mtobjects-root "$MTOBJECTS_ROOT" \
  --max-images 20 --initial-points 12 --max-iter 48 --workers 8

python "Foreground Masking/optimise_toy_objects_MTObjects.py" \
  --manifest "$MANIFEST" --mtobjects-root "$MTOBJECTS_ROOT" \
  --max-images 20 --initial-points 8 --max-iter 32 --workers 8
```

7. Measured production performance

Times below are wall-clock intervals observed in the saved terminal output, including image preparation but excluding VM startup, inter-run transfer gaps and final deallocation. Spike-gate objectives are minimised; toy-object scores are maximised. They measure different criteria and should not be compared as if they shared one scale.

Optimiser	Cases	Evaluations	Wall time	Mean/eval	Best result
SEP Spike Gate	12 usable galaxies	80	1m 51s	1.39s	objective 13.9208
SEP Toy Objects	20 / 120 toys	40	1m 31s	2.28s	score 0.6513
MTObjects Spike Gate	9 usable galaxies	60	27m 07s	27.1s	objective 4.9278
MTObjects Toy Objects	20 / 120 toys	40	43m 36s	65.4s	score 0.1302
Complete suite	Four programs	220	74m 05s	20.2s	all completed

The complete Azure session, including VM start, small gaps between commands, four downloads and deallocation, was approximately 77 minutes. SEP completed in seconds to minutes; MTObjects accounted for about 95% of optimiser wall time.

8. Best parameters and quality indicators

Optimiser	Selected parameters	Key quality indicators
SEP Spike Gate	threshold 2.4133; minarea 3; deblend 17 / 3.89e-5; background 16; filter 1; dilation 2; max area 2328; max elongation 29.97	mean spike coverage 0.845; minimum 0.343; mean masked fraction 6.60%; maximum 14.35%
SEP Toy Objects	threshold 2.0617; minarea 8; deblend 49 / 3.48e-4; background 24; filter 1; dilation 4; max area 5963; max elongation 25.39	toy detection 0.783; mean toy recall 0.730; mean masked fraction 5.04%; maximum 11.18%
MTObjects Spike Gate	move 0.9196; distance 0.8725; FWHM 4.5685; minarea 5; dilation 8; max area 2698; max elongation 14.66	mean spike coverage 0.700; mean masked fraction 8.71%; mean profile affected 22.15%
MTObjects Toy Objects	move 0.7751; distance 0.8291; FWHM 2.1439; minarea 13; dilation 1; max area 1173; max elongation 16.33	toy detection 0.175; mean toy recall 0.146; mean masked fraction 0.204%; maximum 0.523%

Interpretation: The best MTObjects toy-object solution is conservative: it masks very little background but recovers fewer injected objects than SEP. This is a scientific trade-off rather than evidence that the run failed.

9. Cost estimate

The Azure portal displayed US$0.174 per hour for the 2-vCPU D2ns_v6 size. The 8-vCPU D8ns_v6 size has four times the vCPU count; using a linear same-family estimate gives approximately US$0.696 per VM-hour. Actual Azure billing is authoritative and can vary by region, agreement, currency, taxes and price changes.

Scenario	Estimated VM time	Rate assumption	Estimated compute
This four-optimiser production session	1.28 h	US$0.696/h	about US$0.89
Four stability repetitions of the measured suite	4.94 h optimiser time	US$0.696/h	about US$3.44 plus overhead
Conservative worst case: 16 runs at MTObjects-toy duration	11.63 h	US$0.696/h	about US$8.10 plus overhead

- Managed OS disks continue to incur a small storage charge while the VM is deallocated.
- A public IPv4 address and outbound data transfer can have separate charges depending on configuration and usage.
- Downloading result CSV/JSON/SQLite files is negligible; transferring large FITS archives or many PNGs can be more material.
- Deallocation stops VM compute billing. Auto-shutdown is useful, but explicit deallocation and status verification remain mandatory.

10. Benefits and limitations of cloud execution

Benefits

- Temporary access to more CPU cores without purchasing or maintaining another workstation.
- A clean, reproducible Ubuntu environment independent of desktop applications and interactive sessions.
- The VM can continue after the desktop is disconnected, provided the optimiser is launched robustly (for example with tmux, systemd-run or nohup).
- Easy resizing within quota: the same disk and software environment can move between small compatibility-test and larger production sizes.
- Process-level isolation is well suited to NumPy/SEP/MTObjects work and avoids much of Python's thread-level Global Interpreter Lock constraint.

Limitations and risks

- Quota and regional capacity can prevent a preferred VM family from being created even after Pay-As-You-Go activation.

- Cloud speed-up is bounded by the number of independent images and by per-image variability; one unusually slow image can determine a trial's completion time.
- MTObjects required a native compiler and GSL, so installation is more involved than a pure Python package.
- SSH and public-IP configuration introduce security responsibilities. Restrict port 22 and protect the private key.
- Costs continue if a VM is left allocated. Monitoring, auto-shutdown and explicit deallocation are operational requirements.
- The desktop all-galaxy PNG generation remains a separate, long-running stage and can dominate end-to-end elapsed time after optimisation.

11. Result retrieval and desktop application

Each cloud result folder was copied into the existing desktop project hierarchy so the canonical all-galaxy programs select the newest best-parameter JSON automatically:

- sep spike optimisation\20260730_170532
- sep toy optimisation\20260730_170742
- mtobjects spike optimisation\20260730_170927
- mtobjects toy optimisation\20260730_173647

The desktop sequence is:

9. **Apply SEP Spike Gate.** Run batch_spike_gate_SEP.py across the full 182-galaxy manifest.
10. **Apply SEP Toy Objects.** Run batch_toy_objects_SEP.py across the same manifest.
11. **Apply MTObjects Spike Gate.** Run batch_spike_gate_MTObjects.py; expect substantially longer processing.
12. **Apply MTObjects Toy Objects.** Run batch_toy_objects_MTObjects.py.
13. **Build comparison panels.** Run Utilities/make_all_method_galaxy_comparison_pngs.py to consolidate the newest complete summaries into one comparison PNG per galaxy.

12. Repeatable operating checklist

14. **Before starting Azure.** Confirm the budget alert, VM size/quota, SSH key location, manifest and local destination folders.
15. **Start and verify.** Start the VM and confirm PowerState/running. Connect by SSH and activate the virtual environment.
16. **Check dependencies.** Load Optuna, SEP and all four MTObjects shared libraries before using paid compute time for a full run.
17. **Check data.** Verify that the manifest has 21 lines (one header plus 20 cases) and that every FITS/model path exists.
18. **Run with persistence.** Use eight workers and save terminal output. Prefer tmux or a service wrapper so a local shell closure cannot terminate the run.

19. **Monitor safely.** Inspect Optuna's SQLite trial counts and the worker process table. A quiet terminal alone does not prove that a run is dead.
20. **Validate output.** Require the expected number of COMPLETE trials, a best JSON, configuration, case list, summary and details before copying results.
21. **Download.** Copy each timestamped folder to its matching desktop optimisation parent and validate again locally.
22. **Deallocate.** Only after successful download, deallocate the VM and confirm PowerState/deallocated.
23. **Apply locally.** Run the four all-galaxy batches and then the compositor. Keep terminal logs and summary CSVs with the generated PNGs.

13. Troubleshooting lessons

Symptom	Cause observed	Resolution
VM sizes listed but unavailable	Subscription quota was zero or family unavailable in region	Upgrade subscription, register Microsoft.Compute, inspect Usage + quotas, choose an available family/region.
Quota page showed no data	Compute resource provider was not registered	Register Microsoft.Compute under the subscription, refresh, then filter quotas.
Visible monitor PowerShell died	Nested Bash/Python quoting was re-parsed by PowerShell	Copy a standalone Bash monitor script to the VM and invoke it with one simple SSH command.
No terminal output while process exists	Pipe buffering or a long MTObjects image	Inspect SQLite trial states and CPU use; do not terminate solely because the console is quiet.
PowerShell Import-Csv rejected MTObjects toy summary	The existing CSV contains duplicate parameter columns	Use Python csv/SQLite for validation; the study and best JSON remain valid.
SSH connection closes	Local window/network ended, or VM was deallocated	Use ServerAlive settings and a persistent remote runner; reconnect and inspect the study rather than restarting blindly.

14. Conclusions and next stage

Linux compatibility and eight-worker Azure execution have now been demonstrated for all four optimisers. The cloud workflow is technically viable, reproducible against serial execution, and inexpensive for a single full suite. The principal performance constraint is MTObjects, especially the toy-object optimiser; SEP is comparatively negligible.

- Retain the four separate timestamped result folders as the baseline production run.

- Review the 182-galaxy comparison PNGs before treating any optimiser result as scientifically final.
- For the later stability study, use four independent seeds and compare best parameters, scores and downstream masks—not only objective values.
- Plan the stability budget using both the measured-suite estimate and the conservative all-MTObjects-toy estimate.
- Consider adding a persistent remote launcher and automated download/deallocate wrapper before the next multi-run campaign.

References

- Microsoft Learn, Azure virtual machine states and billing: https://learn.microsoft.com/azure/virtual-machines/states-billing
- Microsoft Learn, Create and manage Azure budgets: https://learn.microsoft.com/azure/cost-management-billing/costs/tutorial-acm-create-budgets
- Microsoft Learn, Azure resource providers and types: https://learn.microsoft.com/azure/azure-resource-manager/management/resource-providers-and-types
- Azure Linux virtual machine pricing: https://azure.microsoft.com/pricing/details/virtual-machines/linux/
- Optuna documentation: https://optuna.readthedocs.io/
- SEP documentation: https://sep.readthedocs.io/
- MTObjects source repository: https://github.com/CarolineHaigh/mtobjects

Systematic documentation review - 16 August 2026

Review classification: Reviewed - historical infrastructure and performance report

Updates made: No historical measurements were altered. A dated scope note was added so the Azure benchmark is not mistaken for the August 2026 local four-fold Toy Objects run.

Current interpretation: Use this document for Azure setup, Linux compatibility, transfer and historical performance only. Use the combined outcome document for the final SEP and MTObjects Toy Objects conclusions.

Authoritative combined outcome: SEP and MTObjects Toy Objects Optimisation Summary and Conclusions.docx. Full-precision results: Foreground Masking Optimisation Results.xlsx.